ASSESSMENT 1: ANALYTICAL PROBLEM SOLVING SOLUTIONS

Course Code: CSA17 | Course Name: Artificial Intelligence

1. Water Jug Problem

Problem Statement: Given a 4-gallon jug and a 3-gallon jug (no measuring markers), find how to get exactly 2 gallons of water into the 4-gallon jug.

State Space Representation: (x, y), where x is the water in the 4-gallon jug (0 to 4), and y is the water in the 3-gallon jug (0 to 3).

Initial State: (0, 0)

Goal State: (2, y) where y can be anything (typically (2, 0)).

- Step 1: Fill the 3-gallon jug: (0, 0) -> (0, 3)
- Step 2: Pour water from 3-gallon to 4-gallon jug: (0, 3) -> (3, 0)
- Step 3: Fill the 3-gallon jug again: (3, 0) -> (3, 3)
- Step 4: Pour from 3-gallon into 4-gallon until 4-gallon is full: (3, 3) -> (4, 2) (Since 4-gallon needs 1 more gallon, 3-gallon drops to 2)
- Step 5: Empty the 4-gallon jug onto the ground: (4, 2) -> (0, 2)
- Step 6: Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug: (0, 2) -> (2, 0)

Final Result: Exactly 2 gallons are achieved in the 4-gallon jug.

2. Mars Rover Analysis

- Percepts: Soil composition data via spectrometer, high-resolution terrain images from navigation cameras, wheel slip/resistance feedback, solar panel energy levels, and atmospheric pressure/temperature.
- Operating Environment: Fully observable for immediate surroundings, but partially observable globally; deterministic for physical motor actions but stochastic due to unpredictable terrain obstacles; sequential; dynamic; single-agent.
- Actions: Drive forward/backward, turn left/right, adjust camera angle, deploy robotic arm, drill into rock samples, store samples in chambers, and transmit data back to Earth.
- Performance Measure: Quantity and quality of scientific samples collected, energy efficiency, safe navigation without getting stuck, and accurate telemetry transmission.
- Suitable Agent Architecture: Model-based reflex agent with state. It needs internal state to track visited locations, battery status, and sample storage capacity while handling partial observability by maintaining an internal map of the terrain.

3. 8-Queens Problem Formulation

- States: Any arrangement of 0 to 8 queens on the chessboard.
- Initial State: No queens on the board.
- Actions: Place a queen in any empty square.
- Transition Model: Returns the board with a queen added to the specified square.
- Goal Test: 8 queens are on the board, and none attack each other (no two queens share the same row, column, or diagonal).
- Search Strategy: Can be solved using Backtracking Search (Depth-First Search variant with constraint propagation) or Min-Conflicts heuristic local search.

4. OLA Cab Booking Problem Agent & Pseudocode

Problem-Solving Agent Type: Goal-based agent (and utility-based for cost/comfort optimization). It evaluates different paths, cab types (mini, micro, sedan, shared, prime), fares, and estimated time of arrival (ETA) to satisfy the user's explicit destination and comfort criteria.

Pseudocode:

```
function OLA-Booking-Agent(user_input) returns selected_cab_and_route
    goal.destination = user_input.destination
    goal.comfort_preference = user_input.cab_type

    available_cabs = GetNearbyCabs(user_input.current_location)
    filtered_cabs = []

    for each cab in available_cabs:
        if cab.type == goal.comfort_preference or cab.type is compatible:
            cost = CalculateFare(cab, goal.destination)
            eta = EstimateTime(cab, goal.destination)
            utility = EvaluateUtility(cost, eta, cab.comfort_rating)
            filtered_cabs.append((cab, utility))

    best_option = SelectMaxUtility(filtered_cabs)
    BookCab(best_option.cab)
    return best_option
end function
```

5. Uniform Cost Search (UCS) on Delivery Network

Graph Structure & Edge Costs: S -> A (cost 1), S -> G (cost 12), A -> B (cost 3), A -> C (cost 1), B -> D (cost 3), C -> D (cost 1), C -> G (cost 2), D -> G (cost 3).

- Step 1 (Initialize): Priority Queue (Path, Cost): [(S, 0)]
- Step 2 (Expand S): Pop (S, 0). Add neighbors A and G -> Queue: [(S->A, 1), (S->G, 12)]
- Step 3 (Expand S->A): Pop (S->A, 1). Add neighbors B (cost 4) and C (cost 2) -> Queue: [(S->A->C, 2), (S->A->B, 4), (S->G, 12)]

- Step 4 (Expand S->A->C): Pop (S->A->C, 2). Add D (cost 3) and G (cost 4) -> Queue: [(S->A->C->D, 3), (S->A->B, 4), (S->A->C->G, 4), (S->G, 12)]
- Step 5 (Expand S->A->C->D): Pop (S->A->C->D, 3). Add G (cost 6) -> Queue: [(S->A->B, 4), (S->A->C->G, 4), (S->A->C->D->G, 6), (S->G, 12)]
- Step 6 (Next Minimum): Reaches destination G via path S -> A -> C -> G with total cost 4.

Optimal Path: S -> A -> C -> G with a minimum total cost of 4.